Enterprise AI Receptionist

Vapi Real-Time Architecture & Implementation Guide

July 2026

Enterprise AI Receptionist — Vapi Real-Time Architecture & Implementation Guide

> Looking for the combined, start-to-finish process covering this **and** the n8n background workflows in one ordered walkthrough? See `MASTER-Implementation-Guide.pdf` in this same folder. This document is the focused, real-time-layer-only deep dive.

Scope: the real-time voice layer only (Section 1 of the platform spec). Background automation (n8n) is a separate, already-built system this document treats as a downstream consumer, not something it re-designs.

Audience: whoever wires this up in Vapi — the dashboard, the API, and the `vapi-realtime-platform/` codebase this guide accompanies.

Companion codebase: `vapi-realtime-platform/` (same repo, sibling folder to this `docs/` directory). Every endpoint, schema, and service described below is already implemented there and passes `npm run typecheck` and `npm test`.

---

1. Executive summary

A caller dials one of 100+ businesses' phone numbers. Vapi answers, transcribes, and runs the conversation. For anything that needs real data — "am I an existing patient," "is 2pm Tuesday free," "book it," "what are your hours," "this is an emergency" — Vapi calls out to **one webhook URL**, shared across every business, backed by a small, stateless, always-on Node.js service (`vapi-realtime-platform/`). That service:

- resolves *which* business/location is calling based on the Vapi assistant/phone number id (never a URL-per-tenant — that doesn't scale to 100+ businesses),
- does one fast, deterministic thing (a cache read, a Pinecone query, a Calendar call, a Postgres write) within a strict latency budget,
- returns a small JSON payload for Vapi's LLM to turn into speech,
- and — critically — **never** makes a conversational LLM call and **never** waits on anything non-essential (CRM sync, SMS, email, Slack, analytics). Those are fired at n8n and forgotten.

Background automation (transcripts, CRM, confirmations, analytics, follow-ups) is n8n's job, triggered by this service but never awaited by it. That boundary — real time vs. background — is the single most important architectural line in this whole platform, and it is enforced in code, not just convention (see §8, §15).

---

2. Architecture overview

Why this shape, not a monolith and not "n8n does everything": the constraint that dominates every decision here is Vapi's own latency budget — humans expect a reply within roughly 500–800ms of finishing a sentence. Every hop in that budget (STT finalization, tool round-trip, LLM completion, TTS) is additive. A tool round-trip through n8n (HTTP → n8n queue → node execution → HTTP back) reliably costs 1–3 seconds even with a well-tuned instance; a tool round-trip to a small stateless Node service co-located with a Redis/Postgres in the same region costs tens of milliseconds. That difference is the entire reason this document exists.

---

3. Key architectural decisions (why, alternatives, trade-offs)

3.1 Node.js + TypeScript + Fastify for the real-time layer

- **Why:** Fastify's own overhead is sub-millisecond per request; its schema-based validation (here done via zod, functionally equivalent) is fast and type-safe; Node's non-blocking I/O model is a good fit for a service that is mostly waiting on network calls (Pinecone, Calendar, Postgres, Redis) rather than doing CPU work. JSON-in/JSON-out webhook handling is Node's home turf.
- **Alternatives considered:** Python/FastAPI (comparable async performance, but the team's Pinecone/embedding tooling and Vapi's own SDKs skew Node/TS; slightly worse cold-start if ever run serverless); Go (best raw latency, but slower to iterate and a smaller pool of engineers who can maintain it for a growing set of verticals). Given this is meant to be

extended by a product team over years, not written once, TypeScript's ecosystem and hiring pool won on maintainability without giving up the latency budget.
- **Trade-off:** Node's single-threaded event loop means a CPU-heavy bug (e.g. a bad regex, a huge JSON payload) can stall the whole process. Mitigated by keeping all hot-path logic I/O-bound (§6) and a hard 5s `requestTimeout`.
- **Scalability:** stateless — horizontally scaled by running more machines behind a load balancer; no session affinity required because tenant context is re-resolved (and cached) per request.
- **Latency:** this is the layer the whole design optimizes for; see §12 for the concrete per-endpoint budget.

3.2 Supabase (Postgres) + Redis, not n8n's own data nodes

- **Why:** the real-time path cannot afford n8n's per-execution overhead for a simple `SELECT`. Postgres gives relational integrity for tenant-scoped data (businesses → locations → calendars/customers/appointments) with Row-Level Security as a second line of defense (§10); Redis gives sub-millisecond reads for the handful of things queried on every single turn (tenant resolution, business hours, FAQ cache).
- **Alternatives:** DynamoDB/Firestore (fine at scale, but relational multi-tenant joins — e.g. "this phone number → this location → this calendar" — are more natural in Postgres); a single Postgres with no cache (simpler, but every `business_hours` and tenant-resolution call would hit Postgres on every utterance across 100+ businesses × thousands of calls/day — unnecessary load and latency for data that changes a few times a year).
- **Trade-off:** two systems to run (Postgres + Redis) instead of one. Justified because the two have fundamentally different jobs — durable system-of-record vs. hot ephemeral cache — and conflating them (e.g. caching in Postgres with a TTL column) reintroduces query latency you're trying to avoid.
- **Scalability:** Supabase scales via read replicas and PgBouncer pooling as tenant count grows; Redis scales via a managed cluster (e.g. Upstash) with the cache being purely derived data — it can be flushed and rebuilt with zero data loss, which is what makes it safe to scale carelessly.

3.3 One shared webhook URL + server-side tenant resolution, not one URL per business

- **Why:** Vapi's tool-call and end-of-call-report payloads both include `call.assistantId` (and usually `call.phoneNumberId`). That is enough to resolve business + location server-side (§4). A URL-per-business scheme would mean provisioning infrastructure changes for every new customer — the opposite of "sell this to 100+ businesses."
- **Alternatives:** per-business subdomains/webhook paths (works, but couples infrastructure changes to sales — onboarding a new business would require a deploy); embedding businessId directly in every tool's parameters and trusting the LLM to pass it correctly (rejected — trusting the LLM with a security/tenant boundary is exactly the kind of prompt-injection-adjacent risk enterprise software should not accept; a hallucinated or manipulated businessId could leak another tenant's data).
- **Trade-off:** requires a `tenantResolver` layer and a cache (Redis) to keep the extra Postgres lookup out of the hot path — implemented in `src/middleware/tenantResolver.ts`.
- **Scalability:** onboarding business #101 is a database insert (`businesses`, `locations`, `assistants`, `phone_numbers` rows) plus one `provisionAssistant.ts` script run — zero infrastructure or code changes.

3.4 Vapi's native Knowledge Base as the primary FAQ path, Pinecone as fallback

- **Why:** Vapi's built-in KB is queried in-process as part of the model's own turn — no extra network hop. `/knowledge/search` (backed by Pinecone) exists for (a) questions the native KB doesn't cover, (b) auditability/versioning of knowledge across 100+ tenants from one place, and (c) reuse by non-voice channels (a future web chat widget hitting the same REST endpoint).
- **Trade-off:** two places knowledge can live (Vapi's KB, Pinecone) is a real maintenance cost. Mitigated by keeping Pinecone as the **single source of truth** — the same ingestion pipeline (n8n's `10-kb-ingestion.json`, generalized per business) populates both Vapi's KB (via its Files API) and Pinecone, so there is one authoring workflow, two query paths.
- **Latency:** native KB ≈ 0 extra hops; `/knowledge/search` fallback budget is 450ms (embedding + Pinecone query), circuit-broken and cached (§6, §13).

3.5 Fly.io (always-on) over serverless (Lambda/Cloud Run)

- **Why:** a cold start (100ms–several seconds depending on runtime/package size) landing inside a live call's tool-call round trip would blow the 500–800ms budget outright, and it would do so unpredictably — the worst kind of latency bug to debug from call-quality complaints alone.
- **Trade-off:** always-on machines cost more at low/no traffic than scale-to-zero serverless. Accepted deliberately: this service's whole reason to exist is to be fast on every single request, including the first one after a quiet period.
- **Scalability:** `min_machines_running = 2` in `fly.toml` for HA; horizontal autoscaling adds machines under load exactly like any stateless HTTP service.

---

4. Multi-tenant & multi-location model

Core entities (full DDL in `src/db/migrations/001_init.sql`):

`businesses` (1) → `locations` (many) → `calendars` (1 per location) / `phone_numbers` (many) `businesses` (1) → `assistants` (usually 1, can be more) → `prompt_versions` (many, one active) `businesses` → `knowledge_documents`, `customers`, `appointments`, `calls`, `emergency_logs`, `handoff_requests`, `audit_logs`, `api_keys`.

Call routing sequence (every tool call, every turn):

```
Caller dials +1-555-0100
      │
      ▼
Vapi resolves phoneNumberId → assistantId (Vapi's own dashboard config)
      │
      ▼
Vapi calls tool → POST /webhooks/vapi
   body.message.call = { id, assistantId, phoneNumberId, customer:{number} }
      │
      ▼
tenantResolver.resolveTenantByAssistantId(assistantId)
   Redis HIT?  ──yes──► { businessId, locationId, vertical, timezone,
   │                      changeCutoffHours, emergencyPolicy }  (< 1ms)
   no
      │
      ▼
Supabase: assistants → businesses (+ locations.is_primary)   (~10-30ms, cold path)
      │
      ▼
cache in Redis (TTL 300s), return context
      │
      ▼
resolveLocationByPhoneNumberId(phoneNumberId) — overrides locationId when
one business/assistant serves multiple locations (e.g. a dental group sharing
one assistant across 5 clinics, routed by which number was dialed)
      │
      ▼
dispatch table calls the right service function with (tenant, args)
```

Why resolve on every call instead of once per call session: Vapi's webhook is stateless from this service's point of view — there is no guarantee tool calls for the same phone call hit the same process, and trusting client-supplied tenant IDs is a security boundary this service does not delegate to the LLM (§3.3). The Redis cache makes "every call" cost effectively nothing after the first.

---

5. Intent routing

Two layers, deliberately not one:

1. **Vapi's own function-calling is the primary intent router.** The system prompt (per vertical, `vapi-templates/prompts/*.md`) describes each tool and when to use it; the LLM decides which tool(s) to call each turn. This is *inside* Vapi's own latency-optimized pipeline — adding a separate "classify first, then route" LLM call in front of it (as the n8n chat-based versions do) would double the number of LLM round trips per turn. For a phone call, that's not an option.
2. **Deterministic guardrails inside each endpoint are the safety net**, because an LLM's routing judgment is not trusted for anything safety- or money-critical:
   - `/knowledge/search` enforces a hard confidence floor (`MIN_CONFIDENCE_SCORE = 0.78`) — below it, the answer is "not sure," never an invented fact, regardless of what the LLM asked for.
   - `/emergency-routing` re-checks the issue summary against a small, vertical-agnostic hard-transfer keyword list (`gas smell`, `chest pain`, `uncontrolled bleeding`, …) — even if the LLM under-classified urgency, a keyword hit forces `live_transfer`.
   - `/calendar/book` and `/calendar/cancel|reschedule` enforce the tentative-only and 48-hour-cutoff rules in code, not in the prompt — the LLM can *say* whatever it wants, but it can never actually get the system to hard-confirm a booking or bypass the escalation window.

Signal (from the LLM's tool choice)	Routed to	Confidence gate	Fallback
"book/change/cancel a time"	`/calendar/*`	availability check is authoritative, not the LLM's guess	conflict → offer alternative; escalation window → staff
"what/when/how much/do you…"	native KB, else `/knowledge/search`	Pinecone score ≥ 0.78	"not sure, I'll have the team follow up" + logged
"hours/open now"	`/business-hours`	n/a (deterministic)	—
emergency language	`/emergency-routing`	keyword override always wins	live transfer or logged+notified
"talk to a person" / frustration	`request_human_handoff` tool	n/a	staff notified via n8n
greetings / chit-chat	LLM only	—	no tool call, no Pinecone query

The last row matters as much as the others: **never query Pinecone or hit a downstream API for small talk.** Every unnecessary tool call is latency and cost with no benefit; the prompts are written so the model only reaches for a tool when the turn actually needs external data.

---

6. The real-time endpoints

Latency budgets below are the *server-side* budget (excludes Vapi's own STT/LLM/TTS time); `KNOWLEDGE_SEARCH_TIMEOUT_MS` / `CALENDAR_TIMEOUT_MS` in `.env.example` enforce them.

6.1 `POST /knowledge/search`

- **Why it exists:** fallback FAQ retrieval when Vapi's native KB doesn't cover a question; single source of truth for retrieval logic reusable outside voice.
- **Vapi calls it:** via the `faq_lookup` tool, only when the model decides its own knowledge/native KB doesn't answer the question — the prompt explicitly says "prefer answering from your own knowledge first."
- **Request:** `{ businessId: uuid, query: string(≤500), category?: string, topK?: 1-10 }`

- **Response:** `{ ok: true, data: { found: bool, answerContext: string, matches: [{text, score, category?, source?}], confidence: number, cached: bool, degraded?: bool } }`
- **Validation:** zod schema (`schemas/knowledge.schema.ts`); businessId must resolve to an active business (else 404).
- **Timeout:** 450ms total (embedding + Pinecone query), circuit-broken.
- **Retry:** none at the HTTP layer (a timeout here degrades gracefully instead — see below); the circuit breaker itself absorbs repeated upstream failures by going to open state after `CIRCUIT_BREAKER_ERROR_THRESHOLD_PCT` (default 50%) of calls fail, so a Pinecone outage fails instantly rather than piling up slow requests.
- **Error handling:** a `TimeoutError` is caught inside `knowledgeService` itself and turned into `{ found: false, degraded: true }` — never a raw 5xx to Vapi. Only a genuine bug propagates as a 500.
- **Caching:** answer text cached in Redis for 10 minutes, keyed by `namespace + sha1(query)` — repeat questions in the same call (or across concurrent calls) skip Pinecone entirely.

6.2 `POST /calendar/check`

- **Why:** read-only availability probe, called *before* the assistant offers a specific time, so it never promises a slot it hasn't verified.
- **Vapi calls it:** via `check_availability` , before `book_appointment` when the caller has named a specific time but the assistant wants to confirm first.
- **Request:** `{ businessId, locationId, startIso, durationMin? }`
- **Response:** `{ available: bool }`
- **Validation:** ISO-8601 datetime check; duration 5–480 min.
- **Timeout:** 900ms (Google Calendar FreeBusy call), circuit-broken per operation type (a burst of booking failures doesn't trip availability checks).
- **Retry:** handled by the breaker's own retry-on-half-open behavior; no application-level retry to avoid double-charging the latency budget.

6.3 `POST /calendar/book`

- **Why:** the only endpoint that writes a calendar event and a DB row — everything else is read-only or log-only.
- **Vapi calls it:** via `book_appointment` , only once name, phone, service, and a specific start time are known (prompt instructs the model to gather all of these conversationally before calling it once, not to chain three separate tool calls per §"minimize sequential tool calls" from the original n8n build).
- **Request:** `{ businessId, locationId, firstName, lastName?, phone, email?, service, startIso, durationMin?, idempotencyKey }` . `idempotencyKey` is built server-side in the webhook dispatcher as `` `${callId}:book_appointment:${startIso}` `` — the caller never has to invent one.
- **Response (success):** `{ status: "tentative", appointmentId }`
- **Response (conflict):** HTTP 409, `{ ok:false, error:{ code:"conflict", retryable:true } }` — the assistant is prompted to offer an alternative time, never to retry the same slot blindly.
- **Validation:** phone normalized to E.164 (`utils/phone.ts`); rejected with 400 if it can't be normalized — never silently drop an unreachable phone number.
- **Idempotency:** `claimIdempotencyKey` (Redis `SET NX`) — a retried tool call (Vapi resending after a network blip) returns the original `"tentative"` result instead of creating a second event.
- **Timeout / retry:** same as §6.2 for the FreeBusy check inside it; the `events.insert` call carries its own circuit breaker.
- **What it never does:** say "confirmed." Every booking is `(TENTATIVE)` in the calendar event title and `tentative` in the DB — staff confirmation is a background/n8n concern (§15), matching the policy already proven out in the n8n Version 4/5 builds this platform supersedes for the real-time path.

6.4 `POST /calendar/cancel` / `POST /calendar/reschedule`

- **Why separate from `/calendar/book` :** different risk profile — these can destroy or move a commitment a customer is relying on, so they carry the business's configurable `change_cutoff_hours` escalation rule (default 48h, per-business in `businesses.change_cutoff_hours`), generalized from the dental-specific 48-hour rule already proven in `version-4-dental-assistant.json` .

- **Vapi calls it:** via `cancel_or_reschedule_appointment`, `action` field distinguishes the two; the underlying calendar operation (`deleteEvent` vs `updateEvent`) is the only difference in `calendarService.ts`.
- **Request:** `{ businessId, locationId, phone, action: "cancel"|"reschedule", newStartIso? (required for reschedule) }`
- **Response:** `{ status: "done" | "escalated", message }`
- **Escalation triggers (any one):** appointment not found for this contact; within `change_cutoff_hours` of the appointment; reschedule requested with no new time given. Escalation writes to `handoff`-style background dispatch (staff notified via n8n) and explicitly does **not** touch the calendar.
- **Validation/timeout/retry:** same phone-normalization and Calendar-API budget as booking; `findEventByContact` (a `calendar.events.list` search) has its own circuit breaker so a bad search doesn't also block booking.

6.5 `GET /business-hours`

- **Why:** every business's "are we open" logic (used to append an after-hours disclaimer, per the pattern already in the n8n build's `is_open_now`) needs to be instant and never itself become the slow part of a turn.
- **Vapi calls it:** via the `business_hours` tool, or the assistant can check it proactively before offering same-day appointments.
- **Request:** `{ businessId, locationId }` (query params)
- **Response:** `{ isOpenNow: bool, timezone, hoursToday, afterHoursMessage }`
- **Caching:** Redis, 300s TTL — this is the single most cache-friendly endpoint in the system (data changes a handful of times a year); cache is invalidated by the admin-side config update path (out of scope here, background/n8n or a future admin dashboard).
- **Timeout:** effectively bounded by the Postgres query alone (~10-30ms uncached); no external API involved.

6.6 `POST /emergency-routing`

- **Why separate from FAQ/calendar:** the one endpoint allowed to tell Vapi to invoke a **live transfer**, so it gets its own audit trail and is never subject to the knowledge-confidence gate (an emergency is always logged, never "unsure").
- **Vapi calls it:** via `log_emergency`, called immediately per the prompt whenever the caller describes an emergency (vertical-specific definition — see `vapi-templates/prompts/*.md`; e.g. "gas smell" for HVAC, "knocked-out tooth" for dental, "imminent court deadline" for a law firm).
- **Request:** `{ businessId, locationId, phone?, issueSummary, severity?, callId? }`
- **Response:** `{ action: "log_and_notify" | "live_transfer", transferNumber?, message }`. `action` is decided by the business's default `emergency_policy` **or** a hard-transfer keyword hit (§5) — the latter always wins, so a business configured for `log_and_notify` still forces a live transfer on "gas smell." Vapi's own native `transferCall` capability performs the actual telephony transfer using the returned number — this service decides *when/where*, never proxies the SIP/PSTN leg itself (reinventing that would add latency and telephony complexity Vapi already solves).
- **Error handling:** the DB write (`emergency_logs`) happens synchronously and first — an emergency record must exist even if the background staff-notify dispatch to n8n later fails.

6.7 `POST /customer-validation`

- **Why:** a fast identity lookup, usable the moment a phone number is known — well before the caller has said what they want — to personalize the conversation ("welcome back, Sam") and pre-fill `patient_type` / `customer_type`.
- **Vapi calls it:** via `lookup_customer`, as soon as phone or email is known.
- **Request:** `{ businessId, phone?, email? }` (at least one required)
- **Response:** `{ found: bool, customer: {...} | null }`
- **Caching:** 120s TTL — short, because a customer record can be created mid-call by a subsequent booking; long enough to dedupe repeat lookups within one call.

---

7. Vapi webhook contract

Single route: `POST /webhooks/vapi`, secret-verified via the `x-vapi-secret` header (constant-time compare against `VAPI_WEBHOOK_SECRET` — see §10).

Message type `tool-calls` (mid-call, latency-critical):

```
{
  "message": {
    "type": "tool-calls",
    "call": { "id": "...", "assistantId": "...", "phoneNumberId": "...",
              "customer": { "number": "+1..." } },
    "toolCallList": [
      { "id": "call_abc123", "function": { "name": "book_appointment",
        "arguments": { "first_name": "Sam", "phone": "416...", ... } } }
    ]
  }
}
```

→ resolved to a `TenantContext`, each tool call dispatched **in-process** (not a loopback HTTP call to this same service's own `/calendar/book` — see §9 for why), response:

```
{ "results": [ { "toolCallId": "call_abc123", "result": "{\"status\":\"tentative\",...}" } ] }
```

Message type `end-of-call-report` (once, after hangup, latency does not matter):

```
{ "message": { "type": "end-of-call-report", "call": {...},
  "durationSeconds": 142, "recordingUrl": "...", "summary": "...",
  "endedReason": "...", "transcript": "..." } }
```

→ one fast `calls` upsert (so a row always exists), then `dispatchToN8n('call-completed'|'missed-call', ...)` — everything else (§15) is n8n's job.

Defensive parsing: both `toolCallList` and `toolCalls`, and both string and object `arguments`, are handled — Vapi's exact payload shape has drifted between API versions before (documented as a caveat in the existing `VAPI-VOICE-SETUP.md`); verify against current docs at setup time.

---

8. Folder structure

```
vapi-realtime-platform/
  src/
    config/env.ts                zod-validated env, fails fast at boot
    db/
      supabase.ts                 service-role client
      migrations/001_init.sql     full schema (§4), RLS enabled
    cache/redis.ts                get/set + idempotency-key claiming
    integrations/
      pinecone.ts                  embeddings + namespaced query, circuit-breaker wrapped
      googleCalendar.ts            freebusy/create/delete/update/find, one breaker per op
      n8nDispatcher.ts             fire-and-forget POST, never awaited by callers
    middleware/
      tenantResolver.ts            assistantId/phoneNumberId → TenantContext (Redis-cached)
      auth.ts                      Vapi secret + API key verification
      errorHandler.ts               uniform error envelope
    routes/                        8 public endpoints + /webhooks/vapi + /healthz
    services/                      business logic per capability
    schemas/                       zod request schemas
    utils/                         phone, timeout/retry, response envelope
  scripts/provisionAssistant.ts    generates + pushes one business's Vapi assistant config
  vapi-templates/
    assistant-config.template.json  generic, vertical-agnostic
    prompts/<vertical>.md           dental, chiropractic, med_spa, physiotherapy,
                                     hvac, plumbing, law_firm
  test/                            vitest unit tests
  Dockerfile, fly.toml             always-on container deploy
```

9. Why standalone REST routes AND an in-process dispatch table

The 8 endpoints in §6 are real, independently callable REST routes — useful for testing, for a future non-Vapi channel (web chat) to reuse the exact same knowledge/calendar logic, and for internal tooling/monitoring per endpoint. But the Vapi tool-call dispatcher (§7) does **not** call them over HTTP — it imports and calls the same underlying `services/*` functions directly in the same process. Issuing a loopback HTTP call to itself for every tool call would add a full serialize/deserialize + TCP round trip that buys nothing (no isolation benefit — it's the same process either way) and costs real milliseconds on every single tool call, multiplied across thousands of calls/day. Both call paths — the public route and the in-process dispatch — share one implementation, so there is exactly one place each business rule (e.g. the 48-hour cutoff) is coded.

10. Security considerations

Layer	Mechanism
Vapi → this service	Shared secret (`x-vapi-secret` header) checked with a constant-time compare (`middleware/auth.ts`) — a timing side-channel on a webhook with no other auth would leak the secret byte-by-byte.
Non-Vapi callers (dashboard, future channels)	Hashed API key (`api_keys` table), never the raw key at rest; scoped to one `business_id` .
Tenant isolation	Every query filters by `business_id` explicitly in application code (defense layer 1); Postgres RLS policies on every tenant table (defense layer 2) so a future non-service-role client can never cross tenants even on an app bug.
Secrets	Never hard-coded; `.env` only, validated at boot by `config/env.ts` ; Google Calendar refresh tokens stored encrypted (placeholder `decrypt()` in `calendarService.ts` — wire to your KMS/Vault before production).
Rate limiting	Per-business token bucket (`@fastify/rate-limit` , keyed on `businessId`) — one tenant's traffic spike can't starve another's latency budget (critical at 100+ tenants on shared infrastructure).
PII	Phone/email are the only PII this service touches directly; call recordings/transcripts live in Vapi/n8n's storage, not duplicated here. Audit every write (`services/auditService.ts`) without blocking the request.
Input validation	zod schemas on every route reject malformed input before it reaches business logic or a downstream API call.
Webhook replay	`end-of-call-report` upserts on `vapi_call_id` (unique) — a redelivered report updates the same row instead of duplicating it.

11. Error handling & graceful degradation

Uniform envelope on every response: `{ ok, data? , error?: { code, message, retryable }, meta? }` (`utils/response.ts`). Central mapping in `middleware/errorHandler.ts` :

Failure	HTTP	Behavior
Validation error (zod)	400	`validation_error` , not retryable
Unknown/inactive business or location	404	`business_not_found`
Downstream timeout (Pinecone/Calendar)	504 (route) / graceful payload (knowledge search specifically)	`downstream_timeout` , retryable
Booking conflict	409	`conflict` , retryable (offer another time)
Missing/invalid webhook secret or API key	401	`unauthorized`
Rate limit exceeded	429 (via `@fastify/rate-limit`)	—
Anything unexpected	500	`internal_error` , retryable

The one rule that matters most for a live call: a caller must never experience silence or a hard failure because a downstream system hiccupped. `/knowledge/search` demonstrates the pattern explicitly — a Pinecone timeout is caught *inside the service function* and turned into `{ found: false, degraded: true }` , which the assistant prompt turns into "I'm not sure, let me have the team follow up" rather than a dead tool call.

12. Latency budget (server-side, excludes Vapi's own STT/LLM/TTS time)

Endpoint	Budget	Enforced by
`/knowledge/search`	450ms	`KNOWLEDGE_SEARCH_TIMEOUT_MS`, circuit breaker
`/calendar/check`, `/calendar/book` (FreeBusy + insert)	900ms	`CALENDAR_TIMEOUT_MS`, circuit breaker per op
`/calendar/cancel`, `/calendar/reschedule`	900ms (search + delete/update)	same
`/business-hours`	~1ms cached / ~20ms uncached	Redis cache, no external API
`/emergency-routing`	~10-30ms (one Postgres insert) + async dispatch	synchronous log write is intentionally tiny
`/customer-validation`	~1ms cached / ~20-30ms uncached	Redis cache
Whole-request ceiling	5000ms	Fastify `requestTimeout` — a hard backstop, never expected to be hit

13. Caching strategy

Cache key	TTL	Why
`tenant:assistant:<assistantId>`	300s	resolved on every tool call; config changes rarely
`tenant:phone:<phoneNumberId>`	300s	multi-location routing
`hours:<locationId>`	300s	queried nearly every turn; changes a few times/year
`kb:<namespace>:<sha1(query)>`	600s	repeat FAQ questions across/within calls
`customer:<businessId>:<phone|email>`	120s	short — a booking can create the record mid-call
`idem:book:<callId>:<tool>:<startIso>`	86400s	Vapi tool-call retry dedupe (not really a "cache," a claim)

All cache invalidation for config-shaped data (hours, tenant mapping) is time-based (short TTL) rather than event-based in v1 — simpler to reason about at 100+ tenants than wiring cache-busting into every admin write path; revisit only if staleness windows become a real complaint.

14. Scalability plan

- **Compute:** stateless Fastify instances behind a load balancer; scale horizontally on CPU/latency (Fly.io autoscaling or equivalent). No sticky sessions needed — tenant context is re-resolved (cheaply, via Redis) per request.
- **Database:** Supabase/Postgres with PgBouncer pooling; add read replicas for reporting/analytics load before it ever competes with the transactional path; partitioning `calls` / `audit_logs` by month once volume warrants it (thousands of calls/day is not yet partition-scale, but the schema doesn't fight it later).
- **Cache:** managed Redis cluster; because everything in it is derived/rebuildable (§13), it can be scaled, flushed, or resized without a migration.

- **Vector search:** one Pinecone **namespace per business** is the v1 design — simple, perfectly isolated, cheap to reason about. At 100+ businesses with wildly different document volumes, watch index-level metadata cardinality; if namespace count or per-namespace vector count becomes a real cost/perf issue, the fallback is **one index per vertical** (dental, HVAC, …) with `business_id` as a metadata filter instead of a namespace boundary — more index management, less namespace sprawl. Not needed at current scale; documented here so the trade-off is a conscious future decision, not a surprise.
- **Telephony/voice:** entirely Vapi's concern to scale; this platform's job is to stay fast and available so it's never the bottleneck.
- **Background automation:** n8n scales independently (queue mode, more workers) with zero coupling to real-time capacity — this is precisely why the two were split in the first place.
- **Multi-region:** if call volume becomes geographically distributed, replicate this service + a Redis cache per region, with Postgres as the cross-region source of truth (accept slightly higher latency on cache misses in secondary regions, or run regional read replicas). Not needed at 100 businesses/thousands of calls a day; a note for the 10x-larger future.

---

15. n8n boundary — the background automation handoff contract

This service never waits on n8n. Every `dispatchToN8n(path, payload)` call (`integrations/n8nDispatcher.ts`) is fire-and-forget, POSTing the **flat** payload each n8n workflow's own `$json.body.*` fields expect — directly, no envelope wrapper — to `${N8N_BASE_URL}/<event-path>` . One event = one n8n webhook = one "[MASTER] ..." workflow in `n8n-workflows/` ; this keeps each workflow's input contract self-documenting (open the workflow, read the Normalize node) instead of requiring a shared schema doc that drifts.

Event path	Fired from	Payload highlights	n8n workflow
`call-completed`	end-of-call-report handler (non-missed calls)	callId, transcript, recordingUrl, customer, sheetId, durationSec	`01-call-completed.json`
`missed-call`	end-of-call-report handler, when `endedReason` indicates voicemail/no-answer	callId, callerPhone, reason, voicemailUrl	`02-missed-call.json`
`appointment-booked`	`bookAppointment` (always tentative)	appointmentId, customer, startTime, serviceType, sheetId	`03-appointment-booked.json` — sends an acknowledgment, never "confirmed"
(staff-triggered, not dispatched by this service)	a GHL automation (optional) or any staff action	same shape as appointment-booked	`04-appointment-confirmed.json` — the only workflow that sends the real "confirmed" message
`appointment-change`	`cancelOrRescheduleAppointment` , both the "done" and "escalated" outcomes	action, status (`done` | `escalated`), reason, newStartTime	`05-appointment-change.json`
`emergency-alert`	`routeEmergency`	issueSummary, severity, action (`log_and_notify` | `live_transfer`)	`06-emergency-alert.json`
`handoff-alert`	`request_human_handoff` tool	reason, callerPhone	`07-handoff-alert.json`
(admin/staff-triggered, not this service)	knowledge document upload	businessId, documentTitle, text/file	`08-kb-upload.json` — also calls back into `/internal/cache/flush` on this service when done
(n8n-scheduled, not this service)	per-business cron (shell workflow)	businessId, followUpDaysAfter, reviewLink	`09-patient-followup.json`

Booking confirmation is deliberately **two separate events**, not one: `appointment-booked` fires the instant the assistant tentatively holds a slot (customer gets "we'll confirm shortly," staff get a review prompt); the actual "you're confirmed" message only goes out later, from workflow 04, when a staff member (or an automation tied to their action, e.g. a GHL pipeline-stage change if a business still uses GHL for pipeline tracking) triggers it. This keeps the "never auto-confirm a phone booking" rule enforced structurally — the automation capable of sending a firm confirmation is simply never wired to anything that fires automatically off the call itself.

This is intentionally the **only** integration surface between the two systems — n8n never calls back into this service synchronously during a live call (the one exception, `/internal/cache/flush`, is a fire-and-forget best-effort cache invalidation with no bearing on call handling), and this service never runs an n8n sub-workflow inline. If n8n is down, calls still get answered, booked, and logged; only the notification/analytics layer is delayed until n8n recovers (its own retry/error-workflow responsibility — see each workflow's `Handle Failure` → `workflow_errors` → `#automation-alerts` chain in `n8n-workflows/`).

---

16. Multi-vertical support

One codebase, one deploy, N verticals — differentiated by:

1. **Vertical prompt file** (`vapi-templates/prompts/<vertical>.md`) — tone, emergency definition, and legal/medical-advice guardrails specific to that industry (a med spa's "emergency" is an allergic reaction; an HVAC company's is a gas smell; a law firm's assistant refuses to discuss case merits at all).
2. **Per-business config row** (`businesses.vertical`, `change_cutoff_hours`, `emergency_policy`) — the same generic tool schema (`lookup_customer`, `book_appointment`, …) behaves differently per business without any code branching on vertical inside `services/*` (only `emergencyService.ts`'s hard keyword list is vertical-aware, and it's additive safety, not business logic).
3. **`scripts/provisionAssistant.ts`** merges (1) and (2) into a concrete Vapi assistant config and pushes it via the Vapi API — onboarding business #101 in a new vertical means writing one new prompt file (if the vertical is new) and running the script, not forking the service.

This is the direct generalization of the dental-only prompt already proven out in `dental-clinic-chatbot/vapi/assistant-config.json` — same tool shape, same golden rules pattern ("never confirm," "never diagnose," "escalate on X"), parameterized per industry instead of hard-coded to dental.

---

17. Step-by-step implementation process

Prerequisites: a Vapi account + API key; Supabase project; Redis instance (Upstash or similar); Pinecone index (dimension must match your embedding model — 1536 for `text-embedding-3-small`, matching the existing n8n KB ingestion); Google Cloud OAuth2 client (Calendar API enabled); an always-on host for this service (Fly.io assumed below, adjust for your choice); the n8n instance you already have built.

1. **Provision data stores.**
   - Run `src/db/migrations/001_init.sql` against your Supabase Postgres.
   - Create the Pinecone index (dimension 1536, cosine) if not already created by the existing n8n ingestion flow — reuse the same index, one namespace per business (namespace = `businesses.slug`).
   - Stand up Redis; note its connection URL.
2. **Configure and deploy `vapi-realtime-platform/`.**
   - `cp .env.example .env`, fill in every value (Supabase, Redis, Pinecone, OpenAI [embeddings only], Google OAuth2 client id/secret, `N8N_BASE_URL`

    - shared secret, `INTERNAL_API_SHARED_SECRET`, and a strong random `VAPI_WEBHOOK_SECRET`).
  - `npm install && npm run build && npm test && npm run typecheck` locally to confirm a clean baseline.
  - Deploy (`fly deploy` with the provided `fly.toml` / `Dockerfile`, or your platform of choice — must be always-on, not scale-to-zero serverless, per §3.5).
  - Confirm `GET https://<your-host>/healthz` returns `200`.

3. **Onboard your first business.**
   - Insert a row into `businesses` (name, slug, vertical, timezone, change_cutoff_hours, emergency_policy).
   - Insert its `locations` row(s) (mark one `is_primary`), `calendars` row (Google Calendar id + OAuth2 refresh token for that calendar), and `phone_numbers` row(s) once you know the Vapi phone number id (step 5).
   - Ingest its knowledge base into Pinecone under namespace = its `slug` (reuse the existing `10-kb-ingestion.json` n8n workflow, pointed at this business's Drive folder / namespace) **and** upload the same documents to Vapi's native Knowledge Base for this business (via the Vapi dashboard or Files API) — see §3.4 for why both.

4. **Generate and push the Vapi assistant.**
   - `VAPI_API_KEY=... REALTIME_API_SERVER_URL=https://<your-host>/webhooks/vapi VAPI_WEBHOOK_SECRET=<same as .env> npx tsx scripts/provisionAssistant.ts <businessId>`
   - This fills `vapi-templates/assistant-config.template.json` with the business's data and its vertical's prompt, POSTs to Vapi, and writes the returned `vapi_assistant_id` back into the `assistants` table.
   - In the Vapi dashboard, spot-check the created assistant: confirm `server.url` and `server.secret` are set, the voice/model look right, and (if used) attach the native Knowledge Base uploaded in step 3.

5. **Connect telephony.**
   - Vapi dashboard → Phone Numbers → import your Twilio (or Vapi-native) number.
   - Assign the assistant from step 4 to that number.
   - Insert the resulting Vapi `phoneNumberId` into this business's `phone_numbers` row (needed for multi-location routing, §4).

6. **Test end-to-end** using the scenarios in §18 before pointing the number at real customers. Watch both Vapi's per-call latency breakdown (dashboard) and this service's logs (structured, per-request, correlated by `callId`) to catch any endpoint blowing its budget (§12) before it's a customer-facing problem.

7. **Repeat steps 3–6 per additional business.** Nothing in steps 1–2 repeats — that's the entire point of the shared-service, tenant-resolved design.

8. **Import the 9 workflows in `vapi-realtime-platform/n8n-workflows/`** (see that folder's own README for the plain-language walkthrough), and confirm `N8N_BASE_URL` on this service points at your n8n instance's webhook base.

---

18. Testing plan

Per-endpoint smoke tests (already scaffolded: `test/phone.test.ts` for the normalization helper both `/calendar/*` and `/customer-validation` depend on — extend with integration tests per service as each business's real Calendar/Pinecone credentials come online). Before going live per business, run through:

1. New caller, no record → `lookup_customer` returns not-found → book an appointment → tentative hold confirmed verbally, never "booked."
2. Same caller calls back → recognized, greeted by name.
3. Ask to book a slot known to be taken → conflict → assistant offers another time, does not double-book.
4. Cancel an appointment inside the cutoff window → escalated to staff, assistant says the team will confirm, does not claim it's done.
5. Cancel an appointment outside the cutoff window → cancelled directly, confirmed.

6. Describe the vertical's emergency scenario (see `vapi-templates/prompts/*.md`) → `log_emergency` fires, correct `action` returned (live transfer vs. log-and-notify) based on both business policy and the hard-keyword override.
7. Ask a question answerable from the native KB → answered with no `/knowledge/search` call (check logs — zero Pinecone hits).
8. Ask a question only in Pinecone, not the native KB → `/knowledge/search` called, confidence gate respected, answer or graceful "not sure."
9. Ask something in neither → "not sure" + logged, not an invented answer.
10. Say "let me talk to a person" → `request_human_handoff` fires.
11. Ask outside business hours → after-hours message appended, request still collected.
12. Force a Pinecone/Calendar outage in a staging environment (revoke a test credential) → confirm graceful degradation, not dead air or a hard error.
13. Retry a `book_appointment` tool call twice with the same `callId` (simulate a Vapi network blip) → confirm exactly one calendar event/DB row, not two.
14. Two different businesses, same test session → confirm tenant isolation: business A's assistant can never retrieve business B's customer or knowledge data (this is the single most important test at 100+-tenant scale).

---

19. Monitoring & observability

- **Structured logs** (pino) on every request, correlated by `callId` / `businessId` — essential once debugging spans 100+ tenants and needs to isolate "which business, which call" fast.
- **Vapi's own dashboard** shows per-turn STT/LLM/TTS/tool-time breakdown — the first place to look when a call "feels slow," to confirm whether the tool round-trip (this service) or Vapi's own pipeline is the culprit.
- **Per-endpoint latency + error-rate metrics** (recommended: OpenTelemetry -> your APM of choice) against the budgets in §12 — alert on P95 breaching budget, not just on hard failures, since a "successful but slow" tool call is exactly the failure mode this whole architecture exists to prevent.
- **Circuit breaker state changes** (opossum emits events) are worth alerting on directly — a breaker tripping open means a real downstream outage, independent of whether individual requests are erroring loudly.

---

20. Future scalability considerations

- **Admin/dashboard product:** a thin authenticated UI over the same Supabase tables (business onboarding, prompt-version management, knowledge upload triggering the same ingestion pipeline) — the RLS policies in §10 are already shaped for a non-service-role client to be added safely.
- **Prompt versioning UI:** `prompt_versions` already supports multiple versions per business with one marked active; a rollback is a single `UPDATE`, no redeploy of this service.
- **Billing/usage metering:** `calls` + `audit_logs` already carry `business_id` and timestamps — a usage-based billing pipeline can aggregate from there without new instrumentation in the hot path.
- **Additional calendar providers** (Calendly, Acuity, non-Google): isolated behind the same `calendarClient` interface shape in `integrations/googleCalendar.ts` — add a sibling module and a `provider` field on `calendars`, no route/service-layer changes.
- **Additional channels** (SMS/web chat reusing `/knowledge/search`, `/customer-validation`): already possible today since those are plain REST endpoints, not Vapi-specific — only the webhook/dispatch layer (§7, §9) is Vapi-shaped.